MANDI BHAV

Project Plan • APIs • Features • Hosting • Evaluation Rubric

Project idea
A simple, Hindi-first mandi-price assistant that helps farmers check verified crop prices, compare markets, and understand the differences—while remaining useful when internet access is weak.

Recommended deployment
Vercel for the frontend + Render for the backend. No Cloudflare.

Build principle: deliver one complete, honest end-to-end demo first. Add advanced features only after the core flow works.

1. Problem Statement

“What Is It Selling For Today?”

Farmers need to know the current selling price of their crops in nearby mandis, but price information can be scattered across websites, reports, and local contacts. It may be difficult to find the right commodity, compare markets, understand when a price was recorded, or use the information when connectivity is poor.

Mandi Bhav aims to make verified mandi-price information easier to find and compare through a clear, bilingual, mobile-friendly experience. Each displayed price should include its market, commodity, unit, date, and source so users can judge how current and trustworthy it is.

2. Goal

- Make prices understandable: show crop, mandi, variety (when available), price per quintal, reported date, and source.
- Help compare: let a farmer compare matching crop/variety records across mandis and see the difference per quintal.
- Work in low-connectivity conditions: cache the app and a clearly dated snapshot of prices for offline lookup.
- Support Hindi and English: use Hindi first, with an English toggle and speech-search where the browser supports it.
- Be honest about data: never invent, interpolate, or forecast prices. Label cached data as old when it is not current.

Success looks like this: a user searches for a crop, sees dated/source-labelled records, compares two mandis, understands the price gap, and can share a short summary.

3. APIs and Browser Capabilities

Use the smallest set of services that can support a working demo. Confirm access, terms, response format, and rate limits before relying on any external endpoint.

API / capability	Purpose	Notes / fallback
AGMARKNET / Agmarknet portal	Primary place to look for official mandi price and arrivals data.	Public portal endpoints may change or block automated requests. Test early; do not assume a stable public API. Keep a validated JSON snapshot as fallback.
data.gov.in API	Alternative source if a suitable agricultural market-price dataset is available.	Dataset-specific availability and API-key requirements must be checked. Record dataset/source and report date.
Google Gemini API	Turn already-retrieved comparison results into a short explanation in Hindi or English.	Call from the backend only; keep the key in a server environment variable. Gemini must not invent prices, calculate the authoritative figures, or forecast prices. Provide a non-AI fallback.
Web Speech API	Speech-to-text search in Hindi (hi-IN) and English (en-IN), when supported.	Browser/device support and permissions vary; typed search must always work. Speech recognition may need internet.
Service Worker + Cache API	Cache the app shell and a dated price snapshot for offline use.	Show last-updated time clearly; never label cached data as live.
Web Share API / Clipboard	Share a price summary or copy SMS-ready text.	If native sharing is unavailable, offer a copy button and a WhatsApp share link with prefilled text.
Geolocation API (optional)	Offer nearby-mandi sorting if the user grants permission and reliable coordinates exist.	Permission is optional; keep coordinates in memory where possible. Straight-line distance must be labelled approximate, not road distance.

Data rule: the frontend may use bundled JSON for a predictable offline demo. Mark demo/sample records honestly, and do not present them as current live market prices.

4. Planned Features

Core features (build these first)

- Bilingual dashboard: Hindi-first UI with a clear English toggle, readable text, large touch targets, and a simple mobile layout.
- Mandi-price cards/table: crop, mandi, variety (if present), min/modal/max or the fields actually provided, unit, reported date, and source.
- Search and filters: commodity/crop, district, and mandi; normalize common spelling/name variants.
- Price comparison: compare matching records and calculate the price gap per quintal with deterministic JavaScript.
- Offline-first PWA: cache the app and a dated JSON snapshot; display a last-updated/data-age indicator and handle no-network states.
- Voice search: Hindi and English speech input where supported, with typed search fallback.

AI-assisted features

- Comparison explainer: Gemini explains the difference using supplied verified records. The app—not the model—does all numeric calculations.
- Shareable digest: generate concise Hindi/English WhatsApp text and SMS-ready copy with the actual date/source; the user sends it manually.

Optional stretch features (only after core demo works)

- Nearby-mandi sorting using reliable coordinates; clearly label straight-line distance as approximate.
- Net-return estimate after the farmer enters a transport cost. Show the formula and inputs.
- Price-quality checks for missing fields, duplicate records, inconsistent units, and suspicious values; flag for review instead of silently correcting.

Build order

1. Dashboard + verified/sample-labelled JSON → 2. Search/filter + comparison math → 3. Offline cache → 4. Voice search → 5. Gemini explanation → 6. Share digest → 7. Nearby sorting and polish.

5. Hosting and Architecture

Selected stack: Vercel + Render. Cloudflare is not part of this plan.

Layer	Service	What it runs
Frontend	Vercel	Static React or HTML/CSS/JavaScript app, dashboard, filters, local calculations, PWA assets and cached JSON.
Backend	Render Web Service	Node.js + Express API. Validates requests, calls Gemini securely, and can proxy external price-data requests if appropriate.
AI	Gemini API	Receives only the relevant verified records and calculated comparison facts; returns an explanation, not a new price source.
Price data	Official source + JSON fallback	Fetch official data if dependable access is confirmed. Keep a validated snapshot in the repo for the offline demo, with a clear timestamp/source label.

Request flow

Farmer's phone → Vercel frontend → Render backend → Gemini API (only when needed). The frontend can still search and compare its cached records without calling the backend.

Deployment checklist

- Push code to GitHub and connect the frontend repository to Vercel.
- Deploy the Node.js/Express API as a Render Web Service.
- Set GEMINI_API_KEY as a Render environment secret; never put it in frontend JavaScript or commit it to Git.
- Configure the frontend's backend URL and allowed CORS origin.
- Test the deployed app on a phone, including a cold backend start, no-network mode, and missing API-key/error cases.

Free-tier caveat: Vercel's Hobby plan is intended for personal/non-commercial use. Render's free web services can spin down after 15 minutes without traffic and may take about a minute to wake up. Confirm current terms and limits before deployment.

Official references

Vercel plans: vercel.com/docs/plans
Render free services: render.com/docs/free
Gemini API key guidance: ai.google.dev/gemini-api/docs/api-key

6. Hackathon Evaluation Rubric

Each category is scored out of 20. The eight categories total 160 points. If a percentage is needed, divide the achieved score by 160 and multiply by 100.

Category	Max	What judges assess	Evidence to show
Problem fit	**20**	Does it answer the problem statement?	Show a farmer can find the crop's recorded mandi price and compare markets.
Innovation	**20**	Original idea or a fresh angle.	Bilingual, voice-first access, offline lookup, transparent data freshness, and actionable comparisons.
Technical depth	**20**	How hard was this to build?	Data normalization/validation, offline PWA, deterministic calculations, secure backend proxy, tests and error handling.
Working demo	**20**	Does it actually run end to end?	Demonstrate search → verified/datetime-labelled results → compare → explain/share; show offline fallback.
Design & UX	**20**	Clear, usable, looks finished.	Readable Hindi-first UI, high contrast, large controls, simple navigation, accessible loading/error states.
Impact & scale	**20**	Who benefits, and could it grow?	Farmers and small traders; extend to more crops/mandis and reliable data sources without changing the core flow.
Presentation	**20**	Clear pitch, answers questions well.	Explain the problem, live workflow, data source, offline behaviour, limitations, and next steps.
Code & commits	**20**	Readable repo, honest commit history.	Clear README, setup/deployment instructions, modular code, environment example without secrets, meaningful commits, and honest TODOs.
TOTAL	**160**		Target: a reliable end-to-end demo over a long list of half-built features.

Final demo checklist

- At least one realistic, clearly labelled price record with source and date.
- Search/filter and comparison work on the deployed frontend.
- No fabricated or forecast prices; old cached records show their age.
- Gemini outage or missing key does not break search and comparison.
- API key is absent from browser code, Git history, and README examples.
- README states what is implemented, what is mocked/sample data, and what is not built.